Computer Programming 1

Lecture – Strings

Strings and C Strings



- C++ has two ways of representing strings
  - The first comes straight from C and it uses pointers and arrays
  - The second is native in C++ and it uses a specific class, named “string”

- If you write code, of course, you can often choose which one to use
- If you modify existing code, you will come across the first and the second type, so might you need to know how to use both

Strings as Array of chars (C Strings)

- A **(C) string** is an array of chars, the last element of which is the null character ('\0')

  E.g., char str [] = "Hello";
  - It is equivalent to: char str [] = {'H', 'e', 'l', 'l', 'o', '\0'};
  - it contains 6 elements: 5 characters + the null character that is automatically added (i.e., the size of the array is, at least, +1 compared to the number of characters you want to represent)

Note:

- The assignment is not defined on arrays, thus the use of a constant string for specifying the value of a string is only allowed in the initialization

Example above:

{ ../Lab_strings/strings.cc,strings2.cc}

Strings as Array of chars: example (1)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.    int size=5;

1.    char s[] = "Hello";
2.
3.    cout << s << ";" << endl;
4.    for (int i = 0; i < size; i++)
5.      cout << "s[" << i << "] = '" << s[i] << "'\n";

1.    return 0;
2.  }
```

```
$./a.out
Hello;
s[0] = 'H'
s[1] = 'e'
s[2] = 'l'
s[3] = 'l'
s[4] = 'o'
```

{ ../Lab_strings/strings02.cc}

Strings as Array of chars: example (1)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.    int size=5;

1.    //char s[] = "Hello";
2.    char s[] = {'H','e','l','l','o','\0'};  //alt. valid initialization
3.
4.    cout << s << ";" << endl;
5.    for (int i = 0; i < size; i++)
6.      cout << "s[" << i << "] = '" << s[i] << "'\n";

1.    return 0;
2.  }
```

```
$./a.out
Hello;
s[0] = 'H'
s[1] = 'e'
s[2] = 'l'
s[3] = 'l'
s[4] = 'o'
```

{ ../Lab_strings/strings02.cc}

Strings as Array of chars: example (1)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.    int size=5;

1.    //char s[] = "Hello";
2.    char s[] = {'H','e','l','l','o',~~'\0'~~};  //alt. valid initialization
3.
4.    cout << s << ";" << endl;
5.    for (int i = 0; i < size; i++)
6.      cout << "s[" << i << "] = '" << s[i] << "'\n";

1.    return 0;
2.  }
```

```
$./a.out
Hello;
s[0] = 'H'
s[1] = 'e'
s[2] = 'l'
s[3] = 'l'
s[4] = 'o'
```

Apparently, we do not observe problems, but you can have problems with several functions.

It is an error not to specify '\0': don't use/write code like this!!

{ ../Lab_strings/strings02.cc}

Strings as Array of chars: example (1)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.    int size=10;

1.    char s[] = "Hello";
2.    //char s[] = {'H','e','l','l','o','\0'};  //alt. valid initialization
3.
4.    cout << s << ";" << endl;
5.    for (int i = 0; i < size; i++)
6.      cout << "s[" << i << "] = '" << s[i] << "'\n";
7.
8.    if (s[5]=='\0')
9.      cout << "\n s[5] is the null character" << endl;
10.
11.   return 0;
12. }
```

```
$./a.out
Hello;
s[0] = 'H'
s[1] = 'e'
s[2] = 'l'
s[3] = 'l'
s[4] = 'o'
s[5] = ''
s[6] = ''
s[7] = ' '
s[8] = 'T'
s[9] = 'k'

s[5] is the null character
```

{ ../Lab_strings/strings02.cc}

Strings as Array of chars: example (2)

```
1.  using namespace std;
2.  #include <iostream>

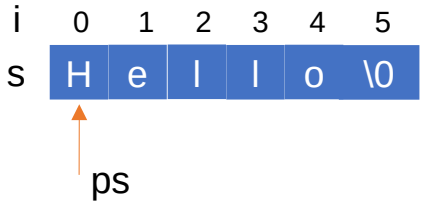
1.  int main()
2.  {
3.    int size=5;

1.    char s[] = "Hello";
2.    //char s[] = {'H','e','l','l','o','\0'};  //alt. valid initialization
3.
4.    cout << s << ";" << endl;
5.    for (int i = 0; i < size; i++)
6.      cout << "s[" << i << "] = '" << s[i] << "'\n";

1.    char* ps = s;
2.    cout << *ps << ";" << endl;
3.    for (int i = 0; i < size; i++)
4.      cout << "s[" << i << "] = '" << *(ps+i) << "'\n";

1.    return 0;
2.  }
```

```
$./a.out
Hello;
s[0] = 'H'
s[1] = 'e'
s[2] = 'l'
s[3] = 'l'
s[4] = 'o'
H;
s[0] = 'H'
s[1] = 'e'
s[2] = 'l'
s[3] = 'l'
s[4] = 'o'
```



{ ../Lab_strings/strings02.cc}

Strings as Array of chars: example (2)

```
1.  using namespace std;
2.  #include <iostream>

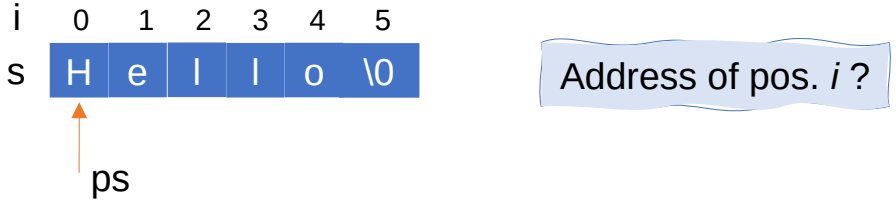
1.  int main()
2.  {
3.    int size=5;

1.    char s[] = "Hello";
2.    //char s[] = {'H','e','l','l','o','\0'};  //alt. valid initialization
3.
4.    cout << s << ";" << endl;
5.    for (int i = 0; i < size; i++)
6.      cout << "s[" << i << "] = '" << s[i] << "'\n";

1.    char* ps = s;
2.    cout << *ps << ";" << endl;
3.    for (int i = 0; i < size; i++)
4.      cout << "s[" << i << "] = '" << *(ps+i) << "'\n";

1.    return 0;
2.  }
```

```
$./a.out
Hello;
s[0] = 'H'
s[1] = 'e'
s[2] = 'l'
s[3] = 'l'
s[4] = 'o'
H;
s[0] = 'H'
s[1] = 'e'
s[2] = 'l'
s[3] = 'l'
s[4] = 'o'
```



{ ../Lab_strings/strings02.cc}

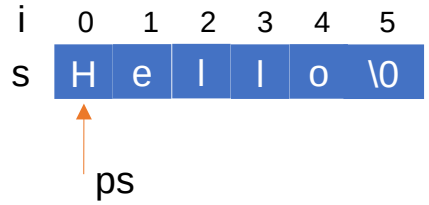
Strings as Array of chars: example (*addresses*)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.   int size=5;
4.   char s[] = "Hello";
5.   char* ps = s;
6.   cout << *ps << ";" << endl;
7.   for (int i = 0; i < size; i++)
8.      cout << "s[" << i << "] = '" << *(ps+i) << "'\n";
9.
10.  cout <<*ps << " " << (void*)ps << " " << (void*)&s[0] << endl;
11.  cout <<*(ps+1) << " " << (void*)(ps+1) << " " << (void*)&s[1] <<
     endl;
12.
13.  return 0;
14.  }
```

```
$./a.out
H;
s[0] = 'H'
s[1] = 'e'
s[2] = 'l'
s[3] = 'l'
s[4] = ‘o’

H 0x7ffedaf09ac2 0x7ffedaf09ac2
e 0x7ffedaf09ac3 0x7ffedaf09ac3
```



Address of pos. *i* ?

- Base address of array + sizeof(char) * index, i.e., *ps+i*
- (void*)ps == (void*)&s[0]
- (void*)(ps+1) == (void*)&s[1]

(recall) (void*) is used to visualize the correct address with char

Note: since ps is a pointer «*&ps*» is the address of the pointer not of the pointer variable

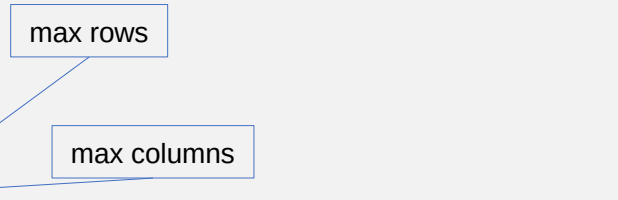
{ ../Lab_strings/strings02.cc}

Strings as Array of chars: example (3a)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.   const int n=4;
4.   const int c=6
5.   char nums[n][c] = { "One", "Two", "Three", "Four" };

1.   for (int i = 0; i < n; i++)
2.      cout << nums[i] << "\n";
3.
4.    return 0;
5.  }
```




```
$./a.out
One
Two
Three
Four
```

{ ../Lab_strings/strings02b.cpp}

Strings as Array of chars: example (2)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.   const int n=4;
4.   const int c=6
5.   char nums[n][c] = { "One", "Two", "Three", "Four" };

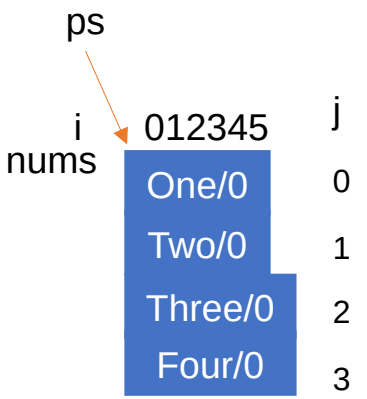
1.   for (int i = 0; i < n; i++)
2.      cout << nums[i] << "\n";

1.   char *ps;
2.   for (int i = 0; i < n; i++){
3.       ps= nums[i];
4.       for (int j = 0; j < c; j++)
5.          cout << *(ps+j);
6.       cout << "\n";
7.    }

1.    return 0;
2.  }
```

{ ../Lab_strings/strings02b.cc}

```
$./a.out
One
Two
Three
Four
One
Two
Three
Four
```



Address of pos. *i* and *i,j* ?

Strings as Array of chars: example (3b)



```
using namespace std;
#include <iostream>

int main()
{
 const int n=4;
 const int c=6
 char nums[n][c] = { "One", "Two", "Three", "Four" };
```

```
cout << long(nums) << endl;                          // 140733823580256
cout << long(&nums) << endl;                 // 140733823580256
cout << nums[0] << endl;                             //One
cout <<  long((void*)&nums[0]) << endl;              // 140733823580256
cout <<  long((void*)&nums[1]) << endl;              // 140733823580262
cout << nums[0][0] << endl;                          //0   ...of One
cout << nums[0][1] << endl;                          //n   ...of One
cout << long((void*)&nums[0][0]) << endl;    // 140733823580256
cout << long((void*)&nums[0][1]) << endl;    // 140733823580257
cout << nums[1][0] << endl;                          //T   ...of Two
cout << long((void*)&nums[1][0]) << endl;    // 140733823580262
cout << long((void*)&nums[1][1]) << endl;    // 140733823580263
```

```
$./a.out
One
Two
Three
Four
…
```

{ ../Lab_strings/strings02b.cpp}

Strings as Array of chars: example (3c)

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int main()
5.  {
6.  char nums[4][6] = { "One", "Two", "Three", "Four" };
7.
8.  cout <<nums[0]<< " &nums[0]="<< long((void*)&nums[0]) << endl;
9.  cout <<nums[1]<< " &nums[1]="<< long((void*)&nums[1]) << endl;
10.
11. cout <<nums[0][0]<< " &nums[0][0]="<< long((void*)&nums[0][0]) << endl;
12. cout <<nums[0][1]<< " &nums[1][1]="<< long((void*)&nums[0][1]) << endl;
13. cout <<nums[1][0]<< " &nums[0][0]="<< long((void*)&nums[1][0]) << endl;
14. cout <<nums[1][1]<< " &nums[1][1]="<< long((void*)&nums[1][1]) << endl;
15.
16. cout << "Address(nums[i=0][j=1])="<< long((void*)&nums)+(0*c+1) << "\n";
17. cout << "Address(nums[i=1][j=1])="<< long((void*)&nums)+(1*c+1) << "\n";
18.
19.   return 0;
20. }
```

```
$./a.out
One &nums[0]=140725201988480
Two&nums[1]=140725201988486

O &nums[0][0]=140725201988480
n &nums[0][1]=140725201988481
T &nums[1][0]=140725201988486
w &nums[1][1]=140725201988487

Address(nums[i=0][j=1])=140725201988481
Address(nums[i=1][j=1])=140725201988487
```

+6 (= num. columns=

= &nums[0]

= &nums[0]+1

= &nums[0]+6

j	0	1	2	3
i	0123	0123	012345	01234
nums	One/0	Two/0	Three/0	Four/0

j=row
i=column

{ ../Lab_strings/strings02c.cc}

Strings as Array of chars: example (3c)

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int main()
5.  {
6.  char nums[4][6] = { "One", "Two", "Three", "Four" };
7.
8.  cout <<nums[0]<< " &nums[0]="<< long((void*)&nums[0]) << endl;
9.  cout <<nums[1]<< " &nums[1]="<< long((void*)&nums[1]) << endl;
10.
11. cout <<nums[0][0]<< " &nums[0][0]="<< long((void*)&nums[0][0]) << endl;
12. cout <<nums[0][1]<< " &nums[1][1]="<< long((void*)&nums[0][1]) << endl;
13. cout <<nums[1][0]<< " &nums[0][0]="<< long((void*)&nums[1][0]) << endl;
14. cout <<nums[1][1]<< " &nums[1][1]="<< long((void*)&nums[1][1]) << endl;
15.
16. cout << "Address(nums[i=0][j=1])="<< long((void*)&nums)+(0*6+1) << "\n";
17. cout << "Address(nums[i=1][j=1])="<< long((void*)&nums)+(1*6+1) << "\n";
18.
19.   return 0;
20. }
```

max rows

max columns

```
$./a.out
One &nums[0]=140725201988480
Two&nums[1]=140725201988486

O &nums[0][0]=140725201988480
n &nums[0][1]=140725201988481
T &nums[1][0]=140725201988486
w &nums[1][1]=140725201988487

Address(nums[i=0][j=1])=140725201988481
Address(nums[i=1][j=1])=140725201988487
```

+6 (= num. columns=

= &nums[0]

= &nums[0]+1

= &nums[0]+6

	j	0	1	2	3
	i	0123	0123	012345	01234
nums		One/0	Two/0	Three/0	Four/0

j=row
i=column

Address of pos.*i* ?

- Base address + sizeof(char) * (row)
- e.g., *((void*) &nums)+j*,

Address of pos. *i,j* ?

- Base address + sizeof(char) * (column * num.colums + row)
- *e.g., ((void*) &nums)+(i*num.column+j)*,

{ ../Lab_strings/strings02c.cc}

Strings as Array of chars: example (3d)

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int main()
5.  {
6.  char nums[4][6] = { "One", "Two", "Three", "Four" };
7.  char* ps=nums[0];
8.  cout <<nums[0]<< " &nums[0]="<< long((void*)&nums[0]) << " "<<long((void*)ps) << endl;
9.  cout <<nums[1]<< " &nums[1]="<< long((void*)&nums[1]) << " "<<long((void*)ps)+6 << endl;
10. cout <<nums[0][0]<< " &nums[0][0]="<< long((void*)&nums[0][0]) << " "<<long((void*)ps)+6 << endl;
11. cout <<nums[0][1]<< " &nums[1][1]="<< long((void*)&nums[0][1]) << " "<<long((void*)ps)+7 << endl;
12. cout << "Address(nums[i=1][j=1])="<< long((void*)&nums)+(1*6+1) << " "<<long((void*)ps)+(1*6+1) << "\n";
13. return 0;
14. }
```

```
$./a.out
One &nums[0]=140726102576704 140726102576704
Two &nums[1]=140726102576710 140726102576710
O &nums[0][0]=140726102576704 140726102576704
n &nums[0][1]=140726102576705 140726102576705
Address(nums[i=1][j=1])=140737052563799 140737052563799
```

	j	0	1	2	3
	i	0123	0123	012345	01234
nums		One/0	Two/0	Three/0	Four/0

j=row
i=column

Address of pos.*i* ?
- Base address + sizeof(char) * (row)
- e.g., *((void*) ps)+j*,

Address of pos. *i,j* ?
- Base address + sizeof(char) * (column * num.colums + row)
- *e.g., ((void*) &ps)+(i+num.column+j),*

{ ../Lab_strings/strings02c.cc}

Size of array of chars: length (1)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.    char d[] = {'h', 'e', 'l', 'l', 'o'};
4.    char d1[] = {'h', 'e', 'l', 'l', 'o','\0'};
5.    char d2[] = "hello";
6.
7.    cout << sizeof(d)/sizeof(d[0]) << endl;
8.    cout << sizeof(d1)/sizeof(d1[0]) << endl;
9.    cout << sizeof(d2)/sizeof(d2[0]) << endl;
10.
11.  char nums[4][6] = { "One", "Two", "Three", "Four" };
12.  cout << sizeof(nums)/sizeof(nums[0]) << endl;
13.  cout << sizeof(nums[0])/sizeof(nums[0][0]) << endl;
14.
15.   return 0;
16. }
```

```
$./a.out
5
6
6

4
6
```

sizeof(var): it gives the size of the var in bytes

E.g.,

- sizeof(char)=1
- sizeof(d)=5
- sizeof(d[0])=1
- size(nums)=24
- size(nums[0])=6
- size(nums[0])=1

{

/Lab_strings/string_size.cpp}

Size of array of chars: length (2)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.   char nums[4][6] = { "One", "Two", "Three", "Four" };
4.   int len0=0;
5.    int num_i=sizeof(nums)/sizeof(nums[0]); //4
6.    int num_j=sizeof(nums[0])/sizeof(nums[0][0]); //6
7.
8.    for (int i=0; i<num_i; i++){ //for each string
9.      len0=0;
10.    while(nums[i][len0] != '\0'){  //search till the end of string character
11.      len0 +=1;
12.    }
13.    cout << "nums " << i << " => " << nums[i] << " " <<  len0 << endl;
14.  }
15.
16.  return 0;
17. }
```

```
$./a.out
nums 0 => One 3
nums 1 => Two 3
nums 2 => Three 5
nums 3 => Four 4
```

{
/Lab_strings/string_size.cpp}

Size of array of chars: length (3)




```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.    char d2[] = "hello";
4.    char *ps=d2;
5.
6.    int len1=0;
7.    while(*(ps+len1)!='\0'){
8.      len1 +=1;
9.    }
10.   cout << len1 << endl; //5
11.
12.   return 0;
13. }
```

```
$./a.out
5
```

{
/Lab_strings/string_size.cpp}

Input and Output of strings

The I/O operators >> and << operate directly on strings (either type)

- The entrance operator >>:
  1. reads characters from cin
  2. stores them sequentially until it encounters a space (which is not read)
  3. stores '\ 0' in the string after the last character read
  4. ends the operation
- The exit operator <<:
  - writes the characters of the string in sequence to cout, up to first '\0' (which is not written)

Example:

```
char buffer[256];        //definition of a string of maximum 255 characters (+ the null character)
cin >> buffer;                  //read the string content from the standard input
cout << buffer;     //write the string content to the standard output
```

Strings and i/o: example (1)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.   const int DIM = 80;
4.   char word[DIM];
5.    do {
6.      cin >> word;
7.      cout << "\t\"" << word << "\"\n";
8.    } while (word[0] != '.');

1.    return 0;
2.  }
```

```
$./a.out
test1
        "test1"
test2
        "test2 "
.
        "."
```

{ ../Lab_strings/strings3.cc}

Strings and i/o: example (1)

```
1.  using namespace std;
2.  #include <iostream>

1.  int main()
2.  {
3.   const int DIM = 80;
4.   char word[DIM];
5.    do {
6.      cin >> word;
7.      cout << "\t\"" << word << "\"\n";
8.    } while (word[0] != '.');

1.    while (cin >> word) {
2.      cout << "\t\"" << word << "\"\n";
3.      if (word[0] == '.') break;
4.    }

1.    return 0;
2.  }
```

```
$./a.out
test1
        "test1"
test2
        "test2 "
.
        "."
test1
        "test1"
test2
        "test2 "
.
        "."
```

{
../Lab_strings/strings3_cinloop.cc}

Some useful functions of the <*iostream*> library (2)

By considering **s = string (array of chars), c = char, n = integer**

- **cin.getline(s, n):** read from cin a line in **s** up to the end of the line, for a maximum of ***n-1*** characters (the '\n' is not read and replaced with \0)
  - It returns (an object equivalent to) 0 if it encounters eof or buffer longer than n, cin.fail() set to true
- **cin.get(c):** read a single character from cin and copy it to c (including spaces), '\0' is returned if c is eof
- **cout.put(c):** write the single character c to cout

Strings and getline: example (1a)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cctype>

1.  int main()
2.  {
3.   const int DIM=80;
4.   char line[DIM];
5.    while (cin.getline(line, DIM)) {
6.      cout << "\t[" << line << "]\n";
7.    };
8.    return 0;
9.  }
```

If the entered string is greater than the max allowed DIM, the program will end

```
$./a.out
test1
            [test1]
```

```
$./a.out
test1 test2
            [test1 test2]
```

{ ../Lab_strings/strings4_while.cc}

Strings and getline: example (1b)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cctype>

1.  int main()
2.  {
3.   const int DIM=5;
4.   char line[DIM];
5.    while (cin.getline(line, DIM)) {
6.      cout << "\t[" << line << "]\n";
7.    };
8.    return 0;
9.  }
```

If the entered string is greater than the max allowed DIM, the program will end

```
$./a.out
testtest
$
```

{ ../Lab_strings/strings4_while.cc}

Strings and getline: example (1c)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cctype>

1.  int main()
2.  {
3.  const int DIM=5;
4.  char line[DIM];
5.   while (!cin.fail()) {
6.      cin.getline(line, DIM);
7.       cout << "\t[" << line << "]\n";
8.    }
9.   if (cin.fail()){
10.     cin.clear();
11.     cin.ignore('\n'); //ignore until \n
12.   }
13.   char buf[255];
14.   cin>> buf;
15.   cout<<buf << endl;
16.   return 0;
17. }
```

```
$./a.out
test
        [test]
test test
test
```

```
$./a.out
testtest
        [test]
test test
test
```

{ ../Lab_strings/strings4_while.cc}

Strings and get and put: example (2)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cctype>

1.  int main()
2.  {
3.   const int DIM=80;
4.   char ch, pre = '\0';
5.   while (cin.get(ch)) {
6.     if (pre == ' ' || pre == '\n')
7.        cout.put(char(toupper(ch)));
8.     else
9.        cout.put(ch);
10.    pre = ch;

1.    return 0;
2.  }
```

```
$./a.out
test1
                [test1]
```

```
$./a.out
first
first
second
Second
third
Third
```

{ ../Lab_strings/strings7_while.cc}

Some useful functions of <*cstring*> library

In the following functions s and t are strings (array of char, i.e., c-style) and c is a character:

- **strlen (s):** it returns the length of s
- **strchr (s, c):** it returns a pointer to the first occurrence of c in s, or NULL (c not in s)
- **strrchr (s, c):** as above but for the last occurrence of c in s
- **strstr (s, t):** it returns a pointer to the first occurrence of the substring t in s, or NULL (t not in s)
- **strcpy (s, t):** it copies t to s and returns s
- **strncpy (s, t, n):** it copies n characters of t into s and returns s (if '\0' is not in n, s will be not well formed)
- **strcat (s, t):** it concatenates t at the end of s and returns s
- **strncat (s, t, n):** it concatenates n characters of t at the end of s and returns ('\0' is always contained in s). s must have enough space for the result. If s is not large enough, program behavior is unpredictable;
- **strcmp (s, t):** it returns a negative, null, or positive value if s is alphabetically less than, equal to or greater than t

On Memory Safety

- Functions copying data to memory are, in general, unsafe (strcpy, strncpy, strcat, strncat).
- Make sure the following conditions are met:
  - The source is properly terminated (or you are using the version with “n” in the name, strncpy)
  - The destination is allocated (with an array, new, or malloc)
  - n lower thatn the size of the destination

Strings and strlen: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.   char s[] = "ABC DEFG";
4.    cout << "strlen(\"" << s << "\") = " << strlen(s) << endl;
5.    cout << "strlen(\"\") = " << strlen("") << endl;
6.    char buffer[80];
7.    cout << "Enter string: ";  cin >> buffer;
8.    cout << "strlen(\"" << buffer << "\") = " << strlen(buffer) << endl;

1.    return 0;
2.  }
```

```
$./a.out
strlen("ABC DEFG") = 8
strlen("") = 0
Enter string: test
strlen("test") = 4
```

```
…
Enter string: string2
strlen("string2") = 7
```

```
…
Enter string: test test1
strlen("string2") = 4
```

{ ../Lab_strings/strings13.cc}

Strings and strlen: example (2)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.   char s[] = "ABC DEFG";
4.    cout << "strlen(\"" << s << "\") = " << strlen(s) << endl;
5.    cout << "strlen(\"\") = " << strlen("") << endl;
6.    char buffer[80];
7.    cout << "Enter string: ";  cin >> buffer;
8.    cout << "strlen(\"" << buffer << "\") = " << strlen(buffer) << endl;

1.    return 0;
2.  }
```

```
char d2[] = “Hello";
char *ps=d2;
cout << sizeof(*ps) << endl; //1
size_t len = strlen(ps);
cout << len << endl; //5
```

```
$./a.out
strlen("ABC DEFG") = 8
strlen("") = 0
Enter string: test
strlen("test") = 4
```

```
…
Enter string: string2
strlen("string2") = 7
```

```
…
Enter string: test test1
strlen("string2") = 4
```

{ ../Lab_strings/strings13.cc}

Strings and strchr: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.   char s[] = "The Mississippi is a long river.";
4.   cout << "s = \"" << s << "\"\n";
5.   char* p = strchr(s, ' ');
6.   cout << "strchr(s, ' ') points to s[" << p - s << "].\n";

1.   p = strstr(s, "is");
2.   cout << "strstr(s, \"is\") points to s[" << p - s << "].\n";
3.
4.   p = strstr(s, "isi");
5.   if (p == NULL)
6.       cout << "strstr(s, \"isi\") returns NULL\n";
7.   cout << "p = " << p << endl;

1.   return 0;
2.  }
```

Note: address of the pointer – address of the array of chars

NULL is returned, if the (sub)string is not detected

```
$./a.out
s = "The Mississippi is a long river."

strchr(s, ' ') points to s[3].

strstr(s, "is") points to s[5].

strstr(s, "isi") returns NULL
```

{ ../Lab_strings/strings14.cc}

Strings and strncpy: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {

1.   char s1[] = "ABCDEFG";
2.   char s2[] = "XYZ";
3.   cout << "Before strncpy(s1,s2,2):\n";
4.   cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
5.   cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
6.   strncpy(s1,s2,2);
7.
8.   cout << "After strncpy(s1,s2,2):\n";
9.   cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
10.  cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
11.
12.  return 0;
13. }
```

```
$./a.out
Before strncpy(s1,s2,2):
        s1 = [ABCDEFG], length = 7
        s2 = [XYZ], length = 3
After strncpy(s1,s2,2):
        s1 = [XYCDEFG], length = 7
        s2 = [XYZ], length = 3
```

Copy 2 chars from s2 to s1

{ ../Lab_strings/strings16.cc}

Strings and strncpy: example (2)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {

1.   char s1[] = "ABCDEFG";
2.   char s2[] = "XYZ";
3.   cout << "Before strncpy(s1,s2,4):\n";
4.   cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
5.   cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
6.   strncpy(s1,s2,4);
7.
8.   cout << "After strncpy(s1,s2,4):\n";
9.   cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
10.  cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
11.
12.  return 0;
13. }
```

```
$./a.out
Before strncpy(s1,s2,2):
        s1 = [ABCDEFG], length = 7
        s2 = [XYZ], length = 3
After strncpy(s1,s2,2):
        s1 = [XYZ], length = 3
        s2 = [XYZ], length = 3
```

Copy 4 chars from s2 to s1

{ ../Lab_strings/strings16.cc}

Strings and strncpy: example (3)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {

1.   char s1[] = "ABCDEFG";
2.   char s2[] = "XYZ";
3.   cout << "Before strncpy(s1,s2,10):\n";
4.   cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
5.   cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
6.   strncpy(s1,s2,10);
7.
8.   cout << "After strncpy(s1,s2,10):\n";
9.   cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
10.  cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
11.
12.  return 0;
13. }
```

{ ../Lab_strings/strings16.cc}

```
$ g++ strings16.cc
$./a.out
Before strncpy(s1,s2,2):
        s1 = [ABCDEFG], length = 7
        s2 = [XYZ], length = 3
After strncpy(s1,s2,2):
        s1 = [XYZ], length = 3
        s2 = [XYZ], length = 3
*** stack smashing detected ***: terminated
Aborted
```

Copy 10 chars from s2 to s1

```
$ g++ -fno-stack-protector strings16.cc
$./a.out
Before strncpy(s1,s2,2):
        s1 = [ABCDEFG], length = 7
        s2 = [XYZ], length = 3
After strncpy(s1,s2,2):
        s1 = [XYZ], length = 3
        s2 = [XYZ], length = 3
```

Strings and strcmp: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.   const int size = 80;
4.    char s1[size] = "Paolo";
5.    char s2[size];
6.    do {
7.      cout << "the word to be compared with `Paolo'?: ";
8.      cin >> s2;
9.      int val = (strcmp(s2,s1));
10.    if (val<0)
11.      cout << s2 << " it comes before `Paolo'\n";
12.    else if (val==0)
13.      cout << s2 << " it comes as`Paolo'\n";
14.    else
15.      cout << s2 << " it comes after `Paolo'\n";
16.   } while (!cin.eof());
17.  return 0;
18. }
```

{ ../Lab_strings/strings16.cc}

```
$./a.out
the word to be compred with `Paolo'?: Paul
Paul it comes after `Paolo’

the word to be compred with `Paolo'?: Paolo
Paolo it comes as`Paolo’

the word to be compred with `Paolo'?: 4
4 it comes before `Paolo'
```

strcmp (s, t): it returns a negative, null, or positive value if s is alphabetically less than, equal to or greater than t

Strings and strcmp: example (2)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.   char a[100];
4.   char b[100];
5.
6.    a = “String 1";
7.    b = “String 2";

1.    if (a == b) {
2.       cout << "C++ is no nice and forgiving.";
3.   return 0;
4.  }
```

```
$ g++ wrong_string_comparison.cpp
wrong_string_comparison.cpp: In function ‘int
main()’:
wrong_string_comparison.cpp:8:9: error:
incompatible types in assignment of ‘const char
[10]’ to ‘char [100]’
    8 |     a = "Stringa 1";
      |         ^~~~~~~~~~~
wrong_string_comparison.cpp:9:9: error:
incompatible types in assignment of ‘const char
[10]’ to ‘char [100]’
    9 |     b = "Stringa 2";
      |         ^~~~~~~~~~~
```

Use strcmp (s, t): it returns a negative, null, or positive value if s is alphabetically less than, equal to or greater than t

{ ../Lab_strings/wrong_string_comparison.cpp}

Strings and strcat: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.    char s1[11] = "ABCDEFG";
4.    char s2[4] = "XYZ";
5.
6.    cout << "Before strcat(s1,s2):\n";
7.    cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
8.    cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
9.    strcat(s1,s2);
10.   cout << "After strcat(s1,s2):\n";
11.   cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
12.   cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
13.
14.  return 0;
15. }
```

```
$./a.out
Before strcat(s1,s2):
        s1 = [ABCDEFG], length = 7
        s2 = [XYZ], length = 3
After strcat(s1,s2):
        s1 = [ABCDEFGXYZ], length = 10
        s2 = [XYZ], length = 3
```

Note: *s1* must contain enough space to receive the concatenated string

{ ../Lab_strings/strings17.cc}

Strings and strcat: example (2)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.    char s1[] = "ABCDEFG";
4.    char s2[] = "XYZ";
5.
6.    cout << "Before strcat(s1,s2):\n";
7.    cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
8.    cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
9.    strcat(s1,s2);
10.   cout << "After strcat(s1,s2):\n";
11.   cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
12.   cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
13.
14.  return 0;
15. }
```

```
$ g++ string17.cc
$./a.out
Before strcat(s1,s2):
        s1 = [ABCDEFG], length = 7
        s2 = [XYZ], length = 3
After strcat(s1,s2):
        s1 = [ABCDEFGXYZ], length = 10
        s2 = [XYZ], length = 3
*** stack smashing detected ***: terminated
Aborted
```

Note: *s1* must contain enough space to receive the concatenated string. This is not the case here

{ ../Lab_strings/strings17.cc}

Strings and strcat: example (2)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.    char s1[] = "ABCDEFG";
4.    char s2[] = "XYZ";
5.
6.    cout << "Before strcat(s1,s2):\n";
7.    cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
8.    cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
9.    strcat(s1,s2);
10.   cout << "After strcat(s1,s2):\n";
11.   cout << "\ts1 = [" << s1 << "], length = " << strlen(s1) << endl;
12.   cout << "\ts2 = [" << s2 << "], length = " << strlen(s2) << endl;
13.
14.  return 0;
15. }
```

{ ../Lab_strings/strings17.cc}

```
$ g++ -fno-stack-protector string17.cc
$./a.out
Before strcat(s1,s2):
        s1 = [ABCDEFG], length = 7
        s2 = [XYZ], length = 3
After strcat(s1,s2):
        s1 = [ABCDEFGXYZ], length = 10
        s2 = [XYZ], length = 3
```

Note: *s1* must contain enough space to receive the concatenated string. This is not the case here

Use **strncat(...,...)**

- strncat (s, t, n): it concatenates n characters of t at the end of s and returns ('\0' is always contained in s)

Strings and strncat: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.
4.    char s1[20] = "Hi!";
5.    const char* s2 = “Too long string";
6.    strncat(s1, s2, sizeof(s1));
7.    cout << s1<<endl;
8.
9.   return 0;
10. }
```

```
$ g++ -fno-stack-protector string17_c1.cpp
$./a.out
Hi!Too long string
```

{ ../Lab_strings/strings17_c1.cc}

Strings and strncat: example (2)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.
4.    char s1[5] = "Hi!";
5.    const char* s2 = “Too long string";
6.    strncat(s1, s2, sizeof(s1));
7.    cout << s1<<endl;
8.
9.   return 0;
10. }
```

```
$ g++ -fno-stack-protector string17_c2.cpp
$./a.out
Hi!Too l
```

```
$ g++ string17_c2.cpp
$./a.out
*** stack smashing detected ***: terminated
Aborted
```

It copies 5 characters from s2 to s1, but it already contains 3+1 characters, so letting s1 contain 4+5 characters

{ ../Lab_strings/strings17_c2.cc}

Strings and strcat: example (3)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cstring>

1.  int main()
2.  {
3.
4.    char s1[5] = "Hi!";
5.    const char* s2 = “Too long string";
6.    int leftover = sizeof(s1) - strlen(s1) – 1;  //-1 is for the terminator
7.    strncat(s1, s2, leftover);
8.    cout << s1<<endl;
9.
10. return 0;
11. }
```

```
$ g++ -fno-stack-protector string17_c3.cpp
$./a.out
Hi!T
```

{ ../Lab_strings/strings17_c3.cc}

Strings

- The implementation of a string as an array of chars comes directly from C
- C++ provides also another implementation, using the class “**string**”
- String provides a higher level of abstraction over “**char***”

- Example
  - char * hello = “Hello world!”
  - string hello = “Hello world!”

String object member functions

Member function name	Description
s.append(***str***)	add text to the end of a string
s.compare(***str***)	return -1, 0, or 1 depending on relative ordering
s.erase(***index***, ***length***)	delete text from a string starting at given index
s.find(***str***) ***s***.rfind(***str***)	first or last index where the start of ***str*** appears in this string (returns string::npos if not found)
s.insert(***index***, ***str***)	add text into a string at a given index
s.length() or ***s***.size()	number of characters in this string
s.replace(***index***, ***len***, ***str***)	replaces *len* chars at given index with new text
s.substr(***start***, ***length***) or ***s***.substr(***start***)	the next *length* characters beginning at *start* (inclusive); if *length* omitted, grabs till end of string
…	

https://cplusplus.com/reference/string/string

C++ string objects and string literals

- String literals are just hard-coded string values:
  - E.g., "hello!" "1234" "#nailedit"
  - They have no methods that do things for us
  - Think of them like “integer literals”, i.e., you can’t do "4.add(5);" //no

- String objects are objects with lots of helpful methods and operators:
  - E.g., string s;
  - We can use methods inside the string
    - E.g., string piece = s.substr(0,3);
    - E.g., s.append(t); //or, equivalently: s+= t;

C++ string objects and string literals



- Even though they are different types, you can mix them as long as there is a string object around to be the "brains" of the operation
  - E.g.,
    - string s;
    - s = "hello!" //string knows how to convert literal
    - s = s + "1234"; //string has + defined as concatenation
    - char ch = ‘A’; //a single ASCII character
    - s += ch; //string knows how to interact with char
    - s += ‘A’; //and char literal

Converting from one to another



- Converting **char*** to **std::string**
  - ```
    const char* c_str = "Hello, C++";
    std::string cpp_str = c_str;
    ```
- Converting from **std::string** to **char***
  - ```
    std::string cpp_str = "Hello";
    const char* c_str = cpp_str.c_str(); // Safe for reading
    ```
  - ```
    std::string cpp_str = "Hello";
    char* mutable_ptr = cpp_str.data();
    mutable_ptr[0] = 'J'; // Safe for writing
    ```
    ```